

10 Proven Start Here Prompts

Copy any prompt, fill the [BRACKETS], send. Mined from 500+ hours of top Claude builders.

1

Plan Mode Opener

Boris Cherny (Claude Code's creator) starts every session in plan mode; planning first prevents the #1 source of wasted tokens — Claude going down the wrong path and scrapping work.

I want to build [WHAT YOU WANT TO BUILD]. Before writing any code, stay in plan mode and help me define the problem in detail. Ask me clarifying questions one at a time, surface gaps in my thinking, and don't move on until you are 95% confident you understand exactly what I need. When you're confident, show me a written plan and wait for my approval before touching any code. Ultrathink.

2

Self-Built Tutor Setup

Turns Claude into a self-built tutor and saves the .md as a lean local index it can reference later — the universal onboarding pattern for any new tool.

Spin up a fresh project for me. Read the documentation at [DOCS OR GITHUB URL], analyze all of it, and save a master reference guide as a markdown file in a docs/ folder so you can route back to it later instead of re-researching. Then explain how this tool works to me in plain English like I've never used it, and recommend the simplest tech stack for my specific goal: [YOUR GOAL].

3

Persistent Status Line

Makes invisible token spend visible so you know exactly when to /clear — turns context management from a chore into a passive habit.

Run /statusline and set up a status line that always shows my current directory, the model I'm using, a visual progress bar of my context window usage, and my token count. Make it persistent so it sits below the prompt window in every session from now on.

4

Clear Context Early & Often

Long-context accuracy drops sharply past ~50-60% of the window (context rot); a manual summary-then-clear resets the window without losing the work.

We're getting deep into this session. Give me a full but concise summary of everything we've done, every decision we locked in, the key files, and the exact next steps — written so a brand-new session could pick up right where you left off. I'll run /clear and paste this back in. From now on, prompt me to do this whenever we cross about 50% of the context window.

5

Right-Size the Model

Running 80% of work on a cheaper model instead of high-effort Opus dramatically extends your session limit with no quality loss on the hard parts.

Run `/model` and switch me to 'Opus plan' so planning runs on Opus and execution automatically drops to Sonnet. From now on use Sonnet as the default for coding, Haiku for sub-agents and simple research, and only escalate to Opus for genuinely hard architectural decisions. Confirm what you've set.

6

The Contractor Prompt Upgrade

Vague prompts produce vague results; stating context + constraints + expected outcome (and asking the expert question) counteracts generic AI-slop output.

Treat yourself as a brilliant contractor I just hired who has never seen my project. Here is the context: [BACKGROUND]. Here are my constraints: [CONSTRAINTS]. Here is the exact outcome I expect: [OUTCOME]. Build [SPECIFIC THING] – and before you start, tell me what an expert in [FIELD] would be thinking about that I haven't mentioned.

7

Atomic Task Decomposition

Smaller tasks use less context and produce better output; task decomposition is the single biggest weapon against context rot and beginner overwhelm.

Take this idea – [YOUR BIG IDEA] – and break it into the smallest possible atomic tasks, in build order. Show me the full task list first and wait for my approval. Then we'll tackle exactly one task at a time, and you'll commit to git after each one before moving to the next.

8

Hands-Free Tool Install

Removes all config friction; Claude hedges by offering manual steps, so an explicit 'you do it' override plus the `.env` rule gets a full, secure install.

I want to install [TOOL NAME]. Here is the documentation: [DOCS OR GITHUB URL]. First check what I already have installed, then install everything I need and walk me through any credentials step by step. Do NOT give me manual instructions – do everything for me. Store any API keys in a `.env` file, never in our chat.

9

Don't Be an Accept Monkey

Understanding front-end vs back-end, databases, and deployment is what separates a real builder who can debug from a replaceable accept-everything vibe coder.

Whenever you recommend a tech choice, library, or approach I might not understand, stop and explain it to me in simple terms before we continue – and if I'm still unclear I'll say 'explain it again.' I never need to write code myself, but I do want to understand the building blocks. So as we go, tell me why you're doing it this way.

10

Build Verification Into the To-Do List

Bakes QA into execution so Claude builds-checks-fixes and hands you a V1 that's 90% there instead of 60%, catching bugs you'd otherwise miss.

When you build [WHAT WE'RE BUILDING], add verification steps directly into your to-do list: after building, take a screenshot and check it looks right, then open the browser dev tools / console to confirm there are no errors, then fix anything broken. Don't move to the next to-do until you're 95% confident the current one actually works.